# Learning without an answer column

Our first unsupervised learning algorithm

A retailer has shopping measurements but no trusted customer-segment labels. Can we summarize patterns as groups of similar shopping behavior?

## Supervised versus unsupervised

| Supervised classification | Unsupervised clustering |
| --- | --- |
| Features plus known targets; learn to predict a class. | Features without training targets; search for structure. |
| KNN asks nearby labeled examples to vote. | K-means assigns each point to a learned center. |
| A held-out label allows accuracy. | No true segment labels means no ordinary accuracy. |

K-means is a useful first clustering method, not a way to prove real customer types. It will partition data even when there are no meaningful natural groups.

## The whole learning journey

Read measurements -> prepare the geometry -> choose starting centers -> assign -> average -> repeat -> assess k and usefulness -> freeze -> assign future records.

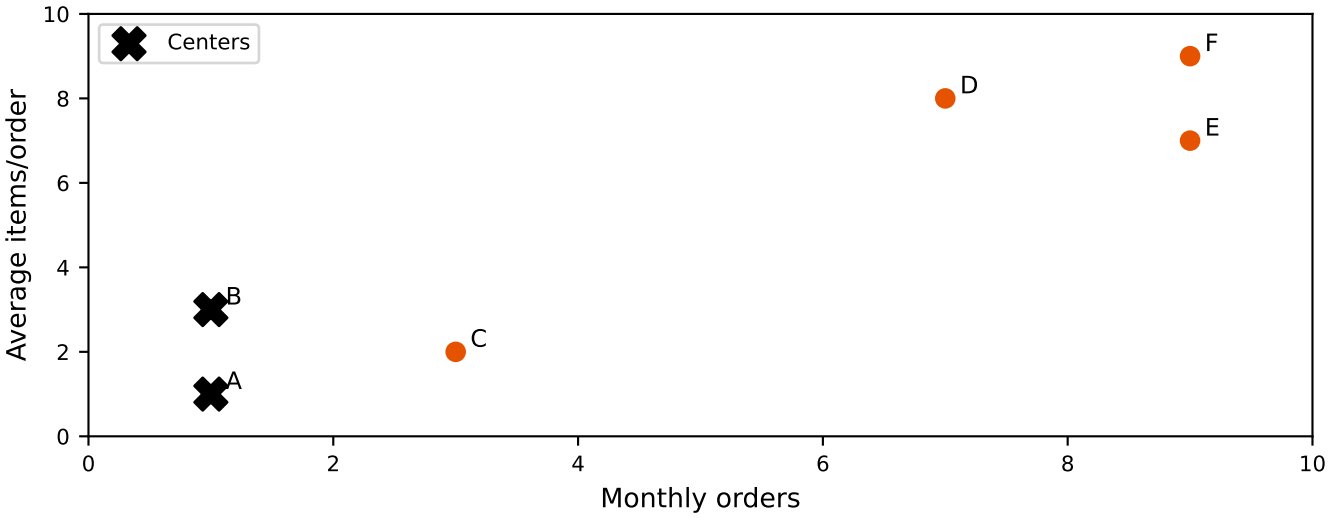
All examples in this lesson are invented. No real customer records or external data are used.

# What do these numbers mean?

Six invented customers; no segment names are supplied

| Customer | Monthly orders | Average items/order |
|---|---|---|
| A | 1 | 1 |
| B | 1 | 3 |
| C | 3 | 2 |
| D | 7 | 8 |
| E | 9 | 7 |
| F | 9 | 9 |

A = (1,1): one order per month, averaging one item per order. E = (9,7): nine orders per month, averaging seven items per order. These are counts, not ratings.




We use raw count coordinates only for this first arithmetic lesson. The larger coding case uses monthly orders and dollar baskets, with training-only scaling. IDs are never features.

# Connect four algorithms you have learned

What does each method learn, and what does a good result mean?

An OBJECTIVE or LOSS is a quantity used to guide fitting. An EVALUATION METRIC assesses a fitted model. Sometimes the same formula serves both purposes, but not every algorithm minimizes a training loss.

| Method | What it uses and learns | Fitting objective / quality check |
|---|---|---|
| Linear regression | Known numerical targets; learns coefficients to predict values. | Mean Squared Error (MSE): average squared difference between actual and predicted targets. |
| Binary logistic regression | Known binary targets; learns coefficients to predict probabilities. | Binary Cross-Entropy (BCE): penalizes probabilities inconsistent with observed binary labels. |
| KNN classification | Known class labels; stores training examples and labels for neighbor voting. | No iterative training-loss minimization in ordinary KNN. Choose settings using validation metrics. |
| K-means clustering | Feature measurements, NO target labels; learns representative cluster centers. | Within-Cluster Sum of Squares (WCSS): total squared distance to assigned centers. |

## Three objectives, not four optimization loops

Regression, logistic regression, and K-means have fitting objectives here. Ordinary KNN does not keep adjusting coefficients or centers until a loss becomes small: fitting mainly retains examples (and may build a search structure).

For KNN, fit preprocessing on training rows, try neighbor counts, distance weights, or other declared settings, and compare validation accuracy, precision, recall, or F1 as appropriate. These evaluate labeled predictions; they do not become an iterative KNN fitting loop.

k means DIFFERENT things: KNN's k counts nearby training examples used to vote; K-means' k counts centers/groups. Neither k is automatically learned by ordinary fitting.

Supervised errors compare with known targets. K-means has no correct segment labels supplied: a compact group is not automatically a true category. Silhouette and stability assess geometry/agreement, not classification accuracy.

# Name the K-means objective: WCSS

Total squared distance to assigned centers; J is only its mathematical symbol

WITHIN-CLUSTER: compare each point with its OWN assigned center. SUM: add contributions from all points. SQUARES: square each coordinate difference before adding. WCSS is the name we will use for this total.

$$J = \text{WCSS} = \sum_{i=1}^{n} \|x_i - \mu_{c_i}\|^2$$

J is a variable name, NOT joules, energy, or a percentage. Other sources call this sum of squared errors (SSE). Scikit-learn calls the fitted total 'inertia' and stores it in inertia_. These are names for the same squared-distance total here.

## Compare what is being squared or penalized

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Regression: if the actual value is 10 and the prediction is 8, that record's squared error is (10-8)^2=4. Average all records' squared errors to obtain MSE.

$$\text{BCE} = -\frac{1}{n} \sum_{i=1}^{n} [y_i \ln(p_i) + (1 - y_i) \ln(1 - p_i)]$$

Binary logistic regression: y is 0 or 1; p is the predicted probability of class 1; ln is the natural logarithm. For y=1, p=0.9 contributes -ln(0.9)=0.1054, while p=0.1 contributes 2.3026. A confident wrong probability is penalized more.

K-means: for point (3,2) assigned to center (1,1), the contribution is (3-1)^2+(2-1)^2=5. Add ALL points' contributions to obtain WCSS; there is no true target y.

## Total versus mean: do not call WCSS 'MSE'

$$\text{mean squared distance} = \frac{\text{WCSS}}{n} \qquad \frac{246}{6} = 41$$

Our first six-customer assignment has WCSS=246 and average squared distance=41 per customer. Each distance already includes both feature columns; we do NOT also divide by the number of columns.

For fixed data and k, dividing by n preserves the best centers. WCSS alone does not choose k. Do not compare numerical losses across different objectives, units, or scales.

# What K-means tries to minimize

k counts centers, not neighbors and not a discovered truth

A feature is an input column. Here the two features are monthly orders and average items per order. The customer ID is not a feature.

### Two centers, each with a pair of numbers

Choose k = 2: ask for TWO centers, not four. Each center needs one number for each input column. That pair locates one point on our two-axis graph.

| One center (one point) | Column 1: orders/month | Column 2: items/order |
|---|---|---|
| Center 0 at the start | 1 | 1 |
| Center 1 at the start | 1 | 3 |

Center 0 is the point (1,1); center 1 is the point (1,3). Two pairs contain four numbers altogether. We do NOT have four centers. Training moves these two points by updating their numbers.

### The goal: make each group compact

$$J = \sum_{i=1}^{n} \|x_i - \mu_{c_i}\|^2$$

Read this as: add every customer's squared distance to its assigned center. J denotes Within-Cluster Sum of Squares (WCSS), called inertia in scikit-learn. x means a customer's measurements; mu means a center; c identifies its assigned center; n counts customers.

You choose the number of centers and the input columns. K-means learns where those centers end up. Assignments follow from the nearest center; meaningful group names are not supplied as training answers.

# Step 1: assign every customer

Start at center 0 = A (1,1), center 1 = B (1,3)

D has 7 monthly orders and 8 items per order: D = (7,8). Subtract matching columns, square each difference, then add. Here d squared means squared distance.

**Compare D with center 0 = (1,1)**

$$d_0^2 = (7-1)^2 + (8-1)^2 = 36 + 49 = 85$$

**Compare D with center 1 = (1,3)**

$$d_1^2 = (7-1)^2 + (8-3)^2 = 36 + 25 = 61$$

61 is smaller than 85, so D goes to center 1. Repeat that same comparison for every row.

| Customer | d squared to 0 | d squared to 1 | Assignment |
|---|---|---|---|
| A | 0 | 4 | 0 |
| B | 4 | 0 | 1 |
| C | 5 | 5 | 0 |
| D | 85 | 61 | 1 |
| E | 100 | 80 | 1 |
| F | 128 | 100 | 1 |

C ties at 5 and 5. Our teaching implementation assigns distance ties to the smaller center index, so C goes to 0. Declare a rule rather than hiding the tie.

Square roots are unnecessary for ranking nonnegative distances: the smaller squared distance also has the smaller distance. K-means optimizes the squared values, not the roots.

Keep just the distance to each row's assigned center, not both distances. The next page shows exactly where all six contributions to J come from.

# Where does J = 246 come from?

Use the starting centers, BEFORE moving either center

J is one total: add the squared distance from EVERY customer to its assigned center. For two input columns, each customer's contribution is:

$$d_i^2 = (x_{i,1} - \mu_{c_i,1})^2 + (x_{i,2} - \mu_{c_i,2})^2$$

In words: (orders minus center orders) squared, plus (items minus center items) squared. Use the center chosen on the previous page.

A and C use center 0 = (1,1). B, D, E, and F use center 1 = (1,3). Below, each line substitutes the customer's coordinates and its assigned center's coordinates, then evaluates both squares.

$$d_A^2 = (1-1)^2 + (1-1)^2 = 0^2 + 0^2 = 0 + 0 = 0$$

$$d_B^2 = (1-1)^2 + (3-3)^2 = 0^2 + 0^2 = 0 + 0 = 0$$

$$d_C^2 = (3-1)^2 + (2-1)^2 = 2^2 + 1^2 = 4 + 1 = 5$$

$$d_D^2 = (7-1)^2 + (8-3)^2 = 6^2 + 5^2 = 36 + 25 = 61$$

$$d_E^2 = (9-1)^2 + (7-3)^2 = 8^2 + 4^2 = 64 + 16 = 80$$

$$d_F^2 = (9-1)^2 + (9-3)^2 = 8^2 + 6^2 = 64 + 36 = 100$$

The first zero belongs to A: A is exactly at center 0. The second belongs to B: B is exactly at center 1. Zero means no distance, not a missing value.

$$J = d_A^2 + d_B^2 + d_C^2 + d_D^2 + d_E^2 + d_F^2$$

$$J = 0 + 0 + 5 + 61 + 80 + 100 = 246$$

Each customer contributes once. Do NOT add both candidate distances, square these results again, or divide by six. J is a total squared error, not an average or a percentage. Updating centers next changes the distances and therefore J.

# Step 2: average assigned records

Hold assignments fixed; calculate a mean for each coordinate

The STARTING centers (1,1) and (1,3) were chosen by using A and B's coordinates. They were not calculated as averages. Now we calculate NEW centers from the assigned customers.

$$\text{new center value} = \frac{\text{sum of values in one column}}{\text{number of customers in this cluster}}$$

Repeat this calculation separately for each input column. Do not mix orders with items, divide by k=2, or divide every cluster by all six customers.

### Center 0: A and C, so divide each column total by 2

A = (1,1), C = (3,2). Their orders are 1 and 3; their items/order values are 1 and 2.

$$\text{orders} = \frac{1+3}{2} = \frac{4}{2} = 2$$

$$\text{items/order} = \frac{1+2}{2} = \frac{3}{2} = 1.5$$

Put those two averages back in column order: new center 0 = (2,1.5). This means 2 orders/month and 1.5 items/order.

### Center 1: B, D, E, F, so divide each total by 4

B = (1,3), D = (7,8), E = (9,7), F = (9,9). The first numbers are orders; the second numbers are items/order.

$$\text{orders} = \frac{1+7+9+9}{4} = \frac{26}{4} = 6.5$$

$$\text{items/order} = \frac{3+8+7+9}{4} = \frac{27}{4} = 6.75$$

New center 1 = (6.5,6.75). A center is an average location: it need not match any actual customer, and fractional values are allowed.

This is why it is called k-MEANS. For fixed assignments, these column averages minimize total squared error.

Using these updated centers with the same assignments lowers J from 246 to 66.25. We are not done: moving centers can change which center is closest.

# Step 3: reassign, then average again

Iteration means one assignment followed by one center update

| ID | d squared to (2,1.5) | d squared to (6.5,6.75) | Group |
|---|---|---|---|
| A | 1.2500 | 63.3125 | 0 |
| B | 3.2500 | 44.3125 | 0 |
| C | 1.2500 | 34.8125 | 0 |
| D | 67.2500 | 1.8125 | 1 |
| E | 79.2500 | 6.3125 | 1 |
| F | 105.2500 | 11.3125 | 1 |

B now belongs to cluster 0. The groups become A,B,C versus D,E,F. The reassignment reduces J to 25.1875 using the old centers.

Each cluster now contains THREE customers, so both column totals are divided by 3. B's move changes both the values being added and the cluster counts.

$$\mu_0 = \left(\frac{1+1+3}{3}, \frac{1+3+2}{3}\right) = \left(\frac{5}{3}, \frac{6}{3}\right) = \left(\frac{5}{3}, 2\right)$$

$$\mu_1 = \left(\frac{7+9+9}{3}, \frac{8+7+9}{3}\right) = \left(\frac{25}{3}, \frac{24}{3}\right) = \left(\frac{25}{3}, 8\right)$$

Example A's contribution uses its assigned center 0:

$$d_A^2 = \left(1 - \frac{5}{3}\right)^2 + (1-2)^2 = \frac{13}{9}$$

$$J = \frac{28}{3} \approx 9.3333$$

J adds all six contributions after the update. It is an error total, not a percentage.

# Read the error totals, not just the table

Same customers A-F: one iteration = assign, THEN update

WCSS after assignment uses the current centers after choosing each customer's nearest group, BEFORE moving centers. WCSS after update keeps those memberships fixed and measures the total again AFTER moving centers to the group means. J is shorthand for WCSS.

## Iteration 1: why does 246 become 66.25?

Starting centers: (1,1) and (1,3). Assign A,C to 0 and B,D,E,F to 1. Then update centers to (2,1.5) and (6.5,6.75). Each table entry is one customer's squared distance to its assigned center.

| Customer | Before centers move | After centers move |
|---|---|---|
| A | 0 | 1.2500 |
| B | 0 | 44.3125 |
| C | 5 | 1.2500 |
| D | 61 | 1.8125 |
| E | 80 | 6.3125 |
| F | 100 | 11.3125 |
| TOTAL WCSS | 246 | 66.2500 |

Example: B stays assigned to center 1 DURING this update. Its new contribution is:

$$d_B^2 = (1 - 6.5)^2 + (3 - 6.75)^2 = 30.25 + 14.0625 = 44.3125$$

B's distance INCREASES from zero, but the TOTAL falls to 66.25. Averaging minimizes the sum for fixed memberships, not every individual's distance.

Iteration 1 ends at J=66.25. Next we reassign customers BEFORE averaging centers again. The next page calculates the new error total; it is not an assumed starting value.

# Iteration 2: reassign, then calculate J

Start from J=66.25; the reassignment produces a NEW total

Keep centers fixed at (2,1.5) and (6.5,6.75). Compare both centers for EVERY customer again. Only B changes its nearest center; the other five customers keep their groups.

### B = (1,3): compare both current centers

$$d^2_{B,1} = (1-6.5)^2 + (3-6.75)^2 = 30.25 + 14.0625 = 44.3125$$

$$d^2_{B,0} = (1-2)^2 + (3-1.5)^2 = 1 + 2.25 = 3.25$$

3.25 is smaller than 44.3125, so B switches from group 1 to group 0. The centers have NOT moved again yet. Only B's contribution to the total changes:

| Customer | Before reassignment | After reassignment |
|---|---|---|
| A | 1.2500 | 1.2500 |
| B | 44.3125 | 3.2500 |
| C | 1.2500 | 1.2500 |
| D | 1.8125 | 1.8125 |
| E | 6.3125 | 6.3125 |
| F | 11.3125 | 11.3125 |
| TOTAL WCSS | 66.2500 | 25.1875 |

### Add the six selected squared distances

$$J = 1.25 + 3.25 + 1.25 + 1.8125 + 6.3125 + 11.3125$$

$$J = 25.1875$$

Equivalently, subtract B's old contribution from 66.25 and add its new one:

$$J = 66.25 - 44.3125 + 3.25 = 25.1875$$

Thus 25.1875 is the total AFTER iteration 2's assignment step, not a number supplied before it. Next, average the new groups A,B,C and D,E,F to move the centers.

# Finish the totals; measure center movement

Error and movement are DIFFERENT quantities

After B switches, the groups are A,B,C and D,E,F. Their means become (5/3,2) and (25/3,8). Using those NEW centers and those fixed groups gives these squared-distance contributions:

| A | B | C | D | E | F |
|---|---|---|---|---|---|
| 1.4444 | 1.4444 | 1.7778 | 1.7778 | 1.4444 | 1.4444 |

$$J = \frac{13+13+16+16+13+13}{9} = \frac{84}{9} = \frac{28}{3} \approx 9.3333$$

This is why iteration 2 shows 25.1875 after assignment and 9.3333 after update. The table displays rounded decimals; the total is calculated with full precision.

### Where does movement = 6.75 come from?

Our teaching code combines ALL center-coordinate changes into one number: square each change, add them, then take the square root. It is not J, a percentage, a sum of the individual centers' travel distances, or the movement of just one center.

| Center | Old position | New position | New minus old |
|---|---|---|---|
| 0 | (1,1) | (2,1.5) | (1,0.5) |
| 1 | (1,3) | (6.5,6.75) | (5.5,3.75) |

$$\text{movement}_1 = \sqrt{1^2 + 0.5^2 + 5.5^2 + 3.75^2} = \sqrt{45.5625} = 6.75$$

For iteration 2, the coordinate changes are (-1/3,1/2) for center 0 and (11/6,5/4) for center 1. Apply the SAME calculation:

$$\text{movement}_2 = \sqrt{\left(-\tfrac{1}{3}\right)^2 + \left(\tfrac{1}{2}\right)^2 + \left(\tfrac{11}{6}\right)^2 + \left(\tfrac{5}{4}\right)^2} \approx 2.2989$$

**Error reductions in order:**

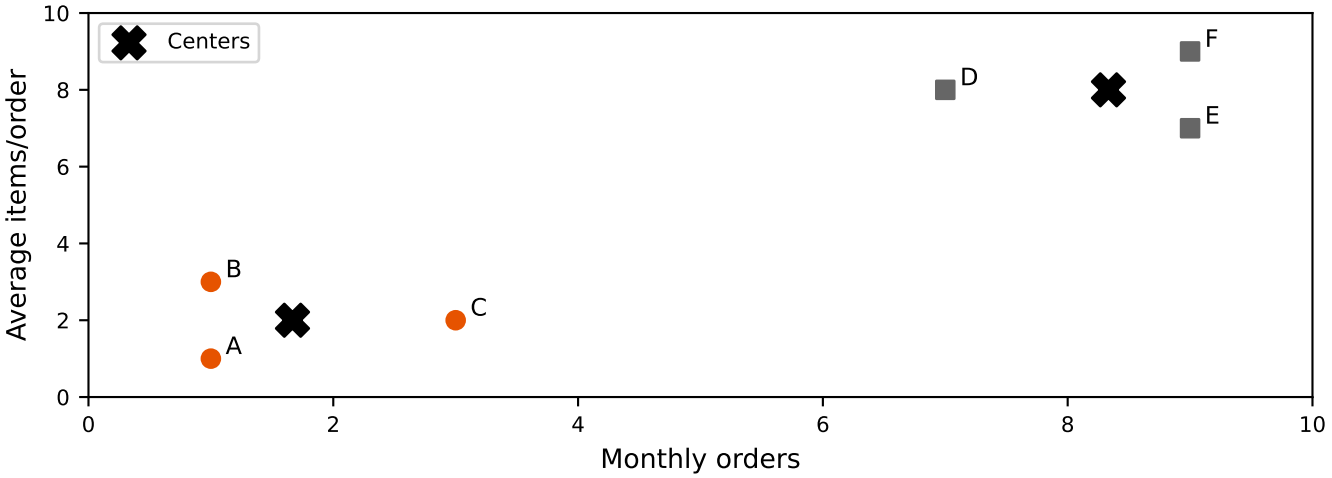
$$246 \longrightarrow 66.25 \longrightarrow 25.1875 \longrightarrow 9.3333$$

Iteration 3: same groups and centers, J=9.3333, movement=0. This run has settled; global optimality is not proven.

# Step 4: check before stopping

A third assignment/update confirms the centers do not change

| Iteration | WCSS after assignment | WCSS after update | Center movement |
|---|---|---|---|
| 1 | 246.0000 | 66.2500 | 6.7500 |
| 2 | 25.1875 | 9.3333 | 2.2989 |
| 3 | 9.3333 | 9.3333 | 0.0000 |




Iteration 3 returns the same assignments and means. Our small implementation stops when center movement is at most 1e-10, or stops at its iteration budget. Here movement is exactly zero.

A maximum-iteration cap is a safety limit, NOT evidence of convergence. Small center movement or unchanged assignments indicates a fixed point of this run, not a globally optimal k or business answer.

# Predicting is not training

Use the learned centers and the same feature transformation

New customer G = (2,2). Reuse the two final centers; do not calculate new ones.

$$d_0^2 = \left(2 - \tfrac{5}{3}\right)^2 + (2-2)^2 = \tfrac{1}{9}$$

$$d_1^2 = \left(2 - \tfrac{25}{3}\right)^2 + (2-8)^2 = \tfrac{685}{9}$$

Assign G to cluster 0. A call to predict does not move either center, refit the scaler, or ask neighbors to vote. The assignment is a nearest-center decision, not a 0-to-1 confidence estimate.

### Pseudocode: training K-means step by step

Pseudocode means plain-language algorithm instructions, not executable Python. Fitting means learning the cluster-center locations from training data.

This standard assign-and-average procedure is called Lloyd's algorithm, named after Stuart Lloyd. It is the SAME procedure we have been using, not a new model or an extra step.

Choose k starting centers.
REPEAT:
Assign EVERY training row to its nearest current center.
AFTER assigning all rows, average each group's rows to update its center.
Check the stopping condition and iteration budget; stop if either is met.
Keep final centers and preprocessing for later assignments.

'Full' means a complete training-data assignment pass BEFORE updating centers. Mini-batch K-means instead updates centers using smaller batches, without waiting for a full assignment pass. Full-data processing can still read rows in blocks while holding centers fixed until the pass ends.

If a cluster receives no rows, its mean is undefined. Production implementations handle this explicitly; our hand-code raises an error and asks for different seeds.

Cluster numbers are only IDs: swapping 0 and 1 changes no grouping. Suggest descriptive names only after reviewing profiles in original units.

# Now you try: four new customers

Work first; do not copy the six-customer answer

| Customer | Monthly orders | Average items/order |
|---|---|---|
| P | 2 | 1 |
| Q | 2 | 3 |
| R | 8 | 7 |
| S | 8 | 9 |

Use k=2, starting centers (2,1) and (8,7). Calculate all squared distances, assign P-S, update both means, then reassign/update until unchanged. Show J before and after each update.

Then assign T=(3,2) without refitting. What was learned? What was chosen? Why is the final distance not accuracy?

Distance table and assignments:

_____

Updated means and J:

_____

Stop check and new-customer explanation:

_____

# Why scale before finding clusters?

NEW four-customer example: not customers A-F or another iteration

We are preparing the measurements BEFORE running K-means. Scaling changes the ruler used for distances; it does not assign groups, move centers, or create buckets.

Each row below is one invented training customer. These FOUR rows are the whole scaling example: no hidden extra rows are needed. The later 600-customer lab is a separate dataset.

| Customer | Orders/month | Average spending/order ($) |
|---|---|---|
| 1 | 2 | 20 |
| 2 | 2 | 40 |
| 3 | 6 | 20 |
| 4 | 6 | 40 |

'Average basket' means the average amount spent on one shopping order. It does NOT mean a bucket, bin, or cluster. For customer 4, (6,40) means six orders/month and average spending of 40 dollars/order.

## Why raw units can accidentally set the priorities

Compare customers 1 and 4. Their orders differ by 4, while spending differs by 20 dollars. With raw numbers:

$$d^2 = (6-2)^2 + (40-20)^2 = 16 + 400 = 416$$

Spending contributes 400 of the 416 squared-distance units; orders contribute only 16. Spending dominates this comparison. That may be intended, but we should not give it extra influence accidentally because of its unit.

If spending were recorded in cents, the same customers would be (2,2000) and (6,4000): the spending contribution would be 2000 squared = 4,000,000. The purchasing behavior did not change; only the unit changed.

## Our choice: express both columns on a common scale

Standardization describes each value relative to its training column's average and spread. It removes the arbitrary dollar-versus-cent scale. It does not guarantee equally useful features or meaningful clusters.

Workflow: raw rows -> learn training means/SDs -> scale rows -> fit K-means.

# SD means standard deviation

A measure of spread, calculated separately for each input column

The MEAN is a column's average. STANDARD DEVIATION (SD) measures how spread out its values are around that average, in the same units as that column. It is not a center, a cluster count, or the average absolute distance.

Use the four training customers from the previous page. First calculate a mean; subtract it from each value; square each difference; average those squares; then take the square root.

**Orders/month: values 2, 2, 6, 6**

$$\text{Orders mean} = \frac{2+2+6+6}{4} = 4$$

$$\text{Orders SD} = \sqrt{\frac{(2-4)^2+(2-4)^2+(6-4)^2+(6-4)^2}{4}}$$

$$= \sqrt{\frac{4+4+4+4}{4}} = \sqrt{\frac{16}{4}} = \sqrt{4} = 2$$

The mean is 4 orders/month; the SD is 2 orders/month. Here every value happens to be 2 orders from the mean. In general, values need not be equally far from their mean.

**Average spending/order: values 20, 40, 20, 40 dollars**

$$\text{Spending mean} = \frac{20+40+20+40}{4} = 30$$

$$\text{Spending SD} = \sqrt{\frac{(20-30)^2+(40-30)^2+(20-30)^2+(40-30)^2}{4}}$$

$$= \sqrt{\frac{100+100+100+100}{4}} = \sqrt{100} = 10$$

The spending mean is 30 dollars/order; its SD is 10 dollars/order. The average squared difference INSIDE each root is called population variance. SD is its square root.

Why divide by 4, not 3? We follow StandardScaler's population-variance convention: divide by the number of training rows. The sample-SD formula taught in statistics often divides by n-1; it is not the convention used here.

# What does a scaled value mean?

Same four customers; change the ruler, not the original shopping behavior

For each column, subtract its TRAINING mean and divide by its TRAINING SD. The result z tells us how many standard deviations above or below the training average that measurement lies.

$$z = \frac{x - \text{training mean}}{\text{training standard deviation}}$$

**Customer 4: 6 orders/month and 40 dollars/order**

$$z_{\text{orders}} = \frac{6-4}{2} = \frac{2}{2} = 1$$

$$z_{\text{spending}} = \frac{40-30}{10} = \frac{10}{10} = 1$$

Thus (6,40) becomes (1,1). This does NOT mean one order and one dollar! Both new values are unitless: each original measurement is one SD above its training average.

| Customer | Raw values | Scaled values | Meaning |
|---|---|---|---|
| 1 | (2,20) | (-1,-1) | Both measurements one SD below average. |
| 2 | (2,40) | (-1,1) | Orders below; spending above average. |
| 3 | (6,20) | (1,-1) | Orders above; spending below average. |
| 4 | (6,40) | (1,1) | Both measurements one SD above average. |

Zero means at the training average, NOT no orders or no spending. Negative values mean below that average; positive values mean above it.

$$d^2_{\text{scaled}} = (1-(-1))^2 + (1-(-1))^2 = 4 + 4 = 8$$

Customers 1 and 4 now differ by two SDs in each column. Each column contributes 4 to their squared distance. Do not compare 8 with raw J as an improvement: we changed the geometry, not the fit.

Save the training means (4,30) and SDs (2,10). Reuse them for validation, test, and future customers; do not fit a new ruler to each pool or each new row. The scaler learns these summaries; K-means then learns centers in the scaled space.

Scaling changes feature weighting. Outliers can distort means, SDs, and centers. A constant column has SD=0; do not divide by zero by hand (StandardScaler uses scale 1 for that column).

# What is learned? What do we choose?

Do not confuse fitting parameters with hyperparameters

| Item | Role and interpretation |
|---|---|
| cluster_centers_ | One row per center, one number per input column. With 2 centers and 2 input columns: 2 rows of 2 numbers. |
| labels_ | Assignments for fitted rows, not observed ground-truth labels. |
| inertia_ | WCSS: total squared distance to assigned centers; lower is more compact. |
| n_iter_ | Iterations used by the retained full-KMeans run. |
| n_clusters (k) | You choose how many groups/centers to ask for. |
| init | How the initial center locations are chosen. |
| n_init | How many complete starting attempts to try; retain the smallest training error. |
| max_iter | Maximum assignment/update rounds for one attempt. |
| tol | How small center movement must be for the tolerance stop check. |
| random_state | Seed to reproduce the random starts; not a quality score. |
| algorithm | Lloyd or Elkan implementation; same squared-Euclidean objective. |

k-means++ spreads initial seeds using distance-based sampling; it does not prove a global optimum. n_init=10 runs full KMeans from ten starts and retains the smallest training inertia.

We use explicit settings instead of version-dependent defaults. Do not select a seed because it gives the most flattering held-out result.

# Choosing k without an answer column

One fit can converge even when the chosen number of groups is poor

Training inertia typically drops as k grows; k equal to the number of distinct rows can memorize them. That does not reveal the correct customer segmentation.

### Combine evidence

Elbow: look for diminishing compactness gains, not a guaranteed sharp corner.
Silhouette: compare within-group versus other-group separation.
Stability: compare repeat fits using the same assessment rows.
Usefulness: review original-unit profiles, group sizes, costs, and actions.

For this lesson, predeclare candidate k=2,3,4,5,6 and choose the highest validation silhouette; exact score ties choose smaller k. This makes the exercise reproducible, not universally optimal.

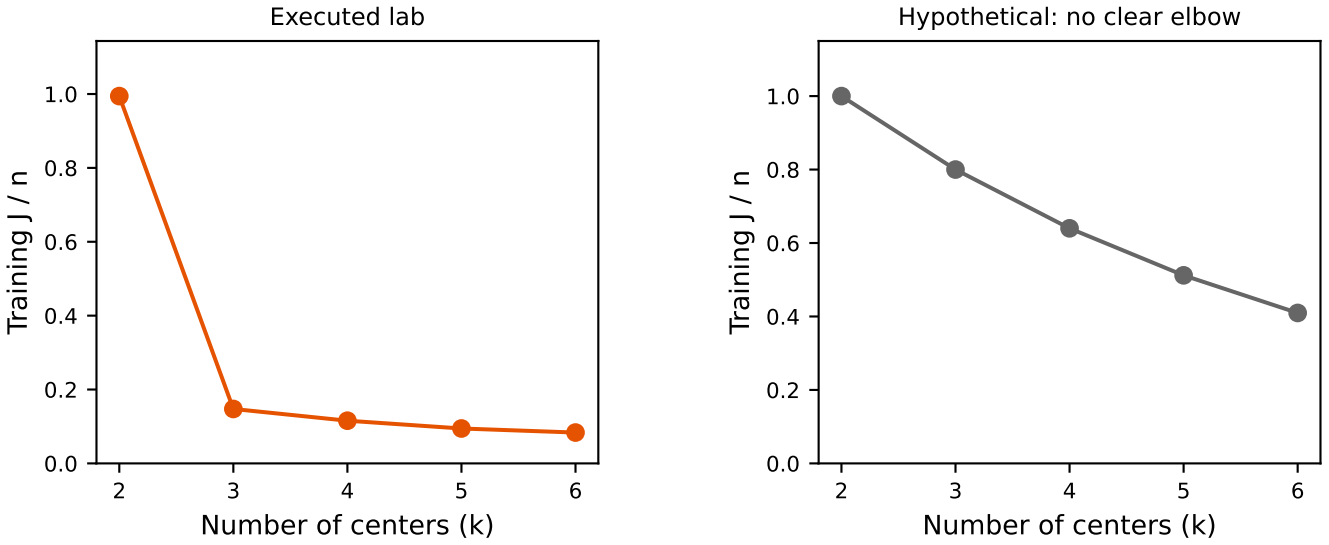
Do not search k until a test score looks good. Do not force a desired story, equate a cluster with a person type, or claim the number of real biological species has been discovered.

Three different decisions: when ONE run stops, which initialization wins, and which k is selected. They happen at different levels.

# Elbow: when do extra centers help less?

Read a change in slope, not just the lowest point

Executed lab: same training rows, training-fitted scaler, and fitting settings for every candidate. We plot J/n so the vertical axis is average squared distance. Lower means more compact, not more correct.




| Change in k | Reduction in training J/n | Relative reduction |
|---|---|---|
| 2 to 3 | 0.8470 | 85.2% |
| 3 to 4 | 0.0318 | 21.6% |
| 4 to 5 | 0.0211 | 18.2% |
| 5 to 6 | 0.0110 | 11.6% |

Read left to right: the large initial drop buys substantial compactness. Later smaller absolute drops buy less. This suggests examining k=3; the elbow alone does not certify it.

The gray hypothetical curve improves smoothly, by 20% each time. It does not show one obvious bend. Its scale is separate from the executed lab; compare the shape, not the heights between panels.

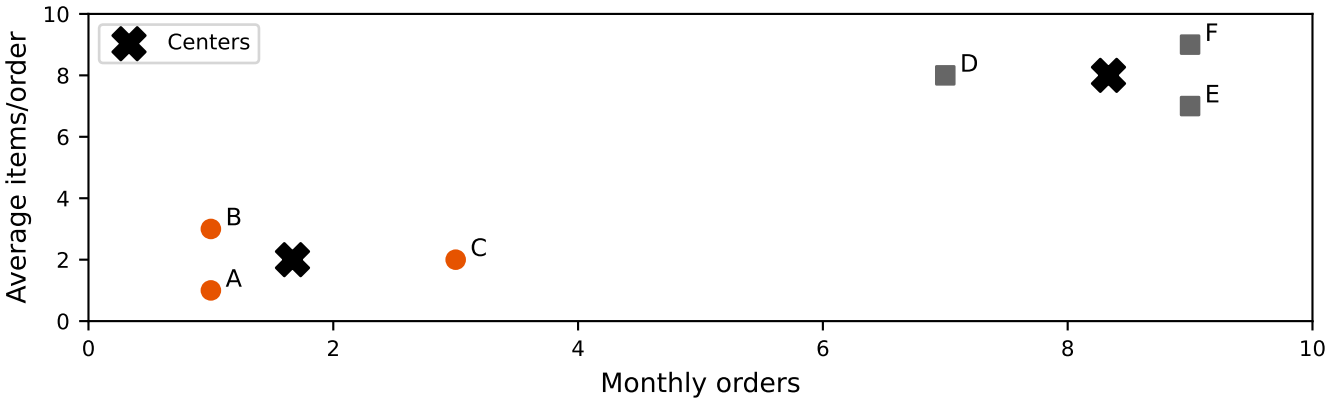
DO: report an ambiguous elbow and combine separation, stability, and usefulness. DO NOT invent a sharp corner, pick k=6 just because it has the lowest error, or compare curves made with different feature scaling.

# What is a silhouette score?

Return to our ORIGINAL six customers A-F, not a new dataset

Silhouette asks: is a customer closer to the OTHER customers in its own group than to customers in competing groups? It assesses an existing grouping. It does not move centers and is not the J that K-means minimizes.

Use our final groups: group 0 = A,B,C; group 1 = D,E,F. The coordinates and assignments below come from the earlier worked example, not an imagined new scenario.




Read the axes: x = monthly orders; y = average items per order. A dot's height is its items/order measurement, NOT its silhouette score. Black X marks show centers, but silhouette uses distances BETWEEN CUSTOMERS.

## Start with A = (1,1): measure to each OTHER customer

Use ordinary Euclidean distance: subtract matching columns, square, add, then take the square root. Unlike J, silhouette here uses distances, NOT squared distances.

| From A to | Expanded ordinary distance | Distance |
|---|---|---|
| B (1,3): own group | sqrt((1-1)^2 + (1-3)^2) = sqrt(0+4) | 2.0000 |
| C (3,2): own group | sqrt((1-3)^2 + (1-2)^2) = sqrt(4+1) | 2.2361 |
| D (7,8): other group | sqrt((1-7)^2 + (1-8)^2) = sqrt(36+49) | 9.2195 |
| E (9,7): other group | sqrt((1-9)^2 + (1-7)^2) = sqrt(64+36) | 10.0000 |
| F (9,9): other group | sqrt((1-9)^2 + (1-9)^2) = sqrt(64+64) | 11.3137 |

sqrt means square root. Exclude A-to-A: a zero self-distance would make its own group look artificially close. Next we average the appropriate distances, not center coordinates.

# Calculate A's silhouette, step by step

Same six customers; all numbers come from the preceding distance table

## 1. Calculate a: average distance to A's own group

A's other group members are B and C: TWO distances, so divide by 2. Lowercase a is a distance average; it is not the customer ID A.

$$a_A = \frac{d(A,B) + d(A,C)}{2} = \frac{2 + \sqrt{5}}{2} \approx 2.1180$$

## 2. Calculate b: closest competing group's average distance

There is only one other group: D,E,F. Average all THREE distances, not just the nearest record. With more groups, calculate a separate average for each competing group and choose the smallest average.

$$b_A = \frac{d(A,D) + d(A,E) + d(A,F)}{3} = \frac{\sqrt{85} + 10 + \sqrt{128}}{3} \approx 10.1778$$

## 3. Compare those averages

$$s_A = \frac{b_A - a_A}{\max(a_A, b_A)} \approx \frac{10.1778 - 2.1180}{10.1778} \approx 0.7919$$

max means use the larger average as the denominator; here b is larger. A is much closer to its own group, giving a positive score near 1. It is NOT a probability or a percentage correct. Calculations retain full precision; displayed distances are rounded.

| Score | Meaning |
|---|---|
| Near +1 | Own group is much closer than competing groups. |
| Near 0 | Own and competing groups are similarly close; inspect overlap. |
| Negative | A competing group is closer on average; inspect the assignment. |

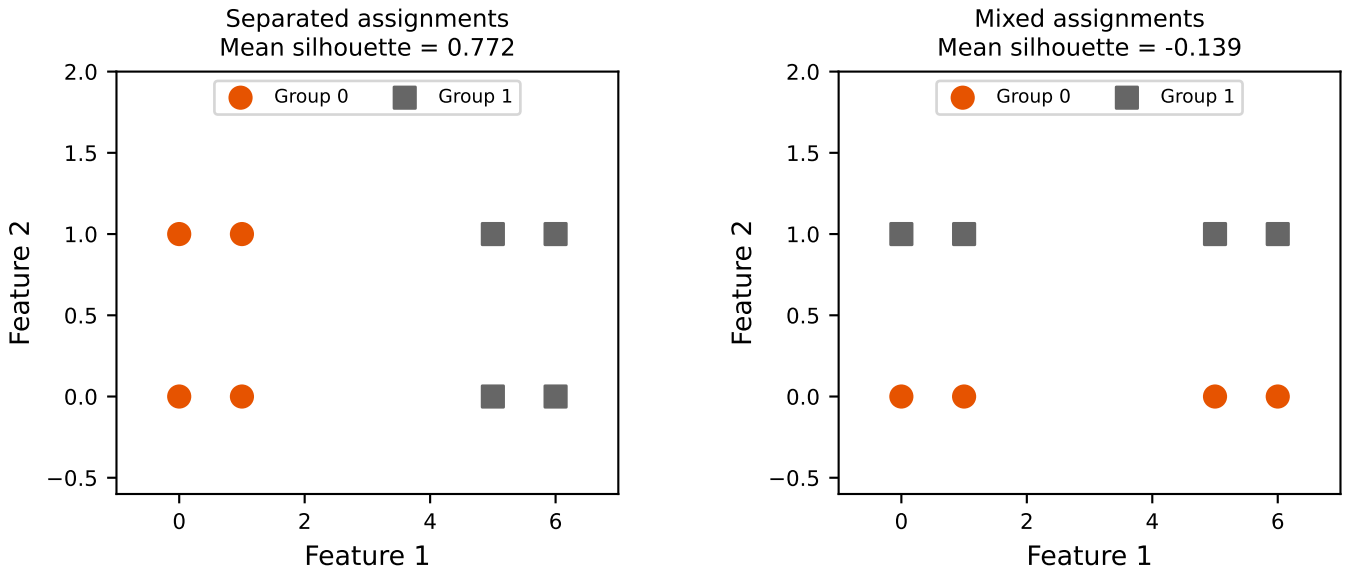
Repeat for every customer, using that customer's own and competing groups:

| A | B | C | D | E | F | Mean of all 6 |
|---|---|---|---|---|---|---|
| 0.7919 | 0.7625 | 0.7233 | 0.7233 | 0.7625 | 0.7919 | 0.7592 |

This mean summarizes the six-customer grouping, not just A. Next we switch explicitly to a NEW eight-point illustration to contrast separated and mixed assignments.

# NEW illustration: eight invented points

Not customers A-F; not the 600-customer lab




The x-axis is an invented first measurement; the y-axis is an invented second measurement, NOT silhouette. Both panels use identical coordinates; only the assigned groups change. Orange circles and gray squares identify groups. We assign these groups by hand, not by running KMeans.

Left: each group occupies one nearby collection of points. Right: each group stretches across both distant locations. The title is the calculated MEAN silhouette over all eight points, not a value plotted on the y-axis.

| Example for ONE record | Calculation | Interpretation |
|---|---|---|
| a=0.4; b=1.0 | (1.0 - 0.4) / 1.0 = 0.6 | Own group is closer on average. |
| a=0.9; b=1.0 | (1.0 - 0.9) / 1.0 = 0.1 | Little separation; inspect boundaries. |
| a=1.0; b=0.4 | (0.4 - 1.0) / 1.0 = -0.6 | Another group is closer on average. |

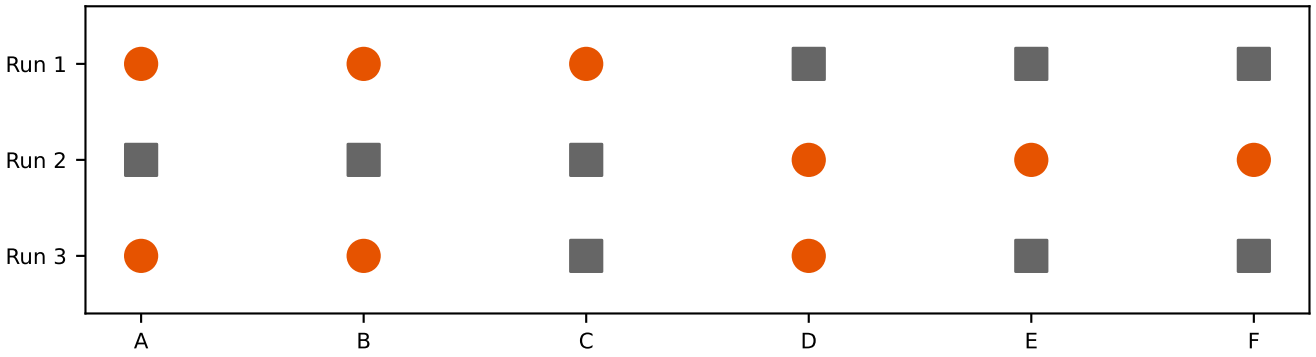
The table above is a SEPARATE hypothetical arithmetic illustration. Its a and b values are supplied, not calculated from either plot or customers A-F. It shows how the formula behaves as own-group and other-group distances change.

DO: inspect low/negative records and per-cluster scores. DO NOT call 0.6 '60% accurate', treat these examples as universal cutoffs, or assume a high average makes every group good.

# Stability: would another start regroup people?

Compare the SAME assessment customers in the SAME row order

Hold training rows, preprocessing, k, and validation customers fixed. Change only the initialization seed, refit on training, then predict the same validation rows. Different customer samples would confound this initialization check.




Same validation customers (hypothetical assignments)

| Comparison | Adjusted Rand index (ARI) | What changed? |
|---|---|---|
| Run 1 vs Run 2 | 1.000 | Only cluster names switched. Group membership is identical. |
| Run 1 vs Run 3 | -0.111 | C and D switch membership; this is a genuinely different grouping. |

ARI compares which pairs stay together. 1 means identical partitions even if IDs are renamed; around 0 means chance-level agreement under its adjustment. Negative values are possible. It is not accuracy against true customer types.

Executed lab: five single-start fits produce ten pairwise validation comparisons. Mean ARI = 1.0000. This easy synthetic case is stable across these starts; that is not proof of natural customer categories.

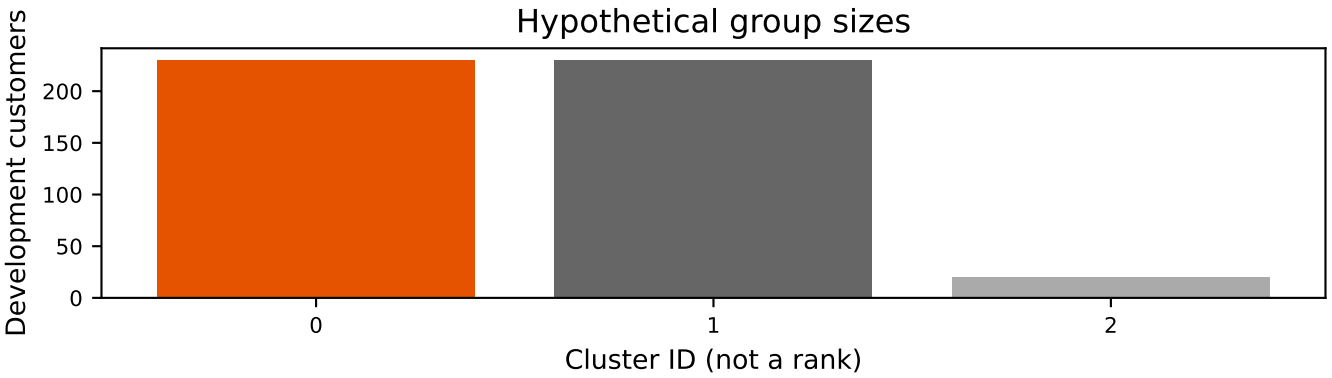
DO: inspect all agreements, use more starts, and investigate competing solutions if agreement is weak. Check sensitivity to training samples separately, using resampled training sets and fixed assessment rows.

DO NOT compare raw cluster ID numbers, pick the seed with the prettiest story, or conclude stable groups must be useful. A stable but arbitrary split can still be a poor business model.

# Usefulness: could we act on these groups?

A good geometry score is a reason to investigate, not a business result

Hypothetical development profiles below: counts sum to 480. These are illustrative, NOT the executed lab's measured profiles. Values are in original units so an operations team can understand them.




| ID / count | Orders/month; basket | Possible next step, not a proven action |
|---|---|---|
| 0 / 230 | 2; $25 | Investigate occasional shopping and whether reminders would help. |
| 1 / 230 | 12; $35 | Investigate frequent small baskets and delivery costs. |
| 2 / 20 | 2; $250 | Inspect large baskets, outliers, and enough evidence for a distinct group. |

DO: check within-group spread, missingness, outliers, coverage, and actionable differences. A 20-person group could be important or noise; size alone does not decide.

$$\text{net incremental value} = 230(0.40 - 1.00) = -138 \text{ dollars}$$

Hypothetical assumption: a proposed contact costs 1 dollar per customer and is expected to add only 0.40 dollars of margin per contacted customer. Contacting all 230 would lose 138 dollars. A high silhouette does not make that action worthwhile.

Treat expected benefit as a hypothesis. Use a controlled pilot to estimate incremental impact and uncertainty before scaling. Review privacy and potential harms.

DO NOT label a cluster 'wealthy' from basket size, infer a person's identity, or claim a campaign caused improvement just because its target cluster looks distinct.

# Three decisions, three different checks

Stopping a fit is NOT the same as selecting a model

| Level | Keep fixed | Decision and evidence |
|---|---|---|
| 1. One starting attempt | Training rows, scaler, k, initial centers | Assign/update until convergence or budget exit. Track center movement, assignments, and J. |
| 2. Several starting attempts | Training rows, scaler, k | For full KMeans, retain the attempt with the smallest training J. n_init sets the number of attempts. |
| 3. Several candidate k values | Training/validation split and scaler | Compare validation silhouette and inspect elbow, stability, and usefulness. Apply a declared selection rule. |

## When can ONE attempt stop?

| Rule | What it means / what to do |
|---|---|
| Assignments unchanged | Same customers stay together; averaging those same groups keeps the centers fixed. |
| Center movement is small | Stop below a declared tolerance. Library-specific scaling matters. |
| Tiny improvement in J | A custom monitor can stop when improvement is small under a declared rule. Standard KMeans does not expose a separate J-improvement threshold. |
| Iteration/time budget | Stop because work is capped. Report the budget exit; do not call it convergence without evidence. |

Our hand-code uses center movement <= 1e-10, with max_iter=20. The full sklearn lab uses max_iter=300 and tol=1e-4; sklearn scales its shift tolerance with training-data variability, not a per-coordinate 0.0001 rule.

Example: a k=2 fit can stop perfectly while merging two useful patterns. A k=6 fit can converge with unnecessarily split groups. Convergence answers 'has this attempt settled?', not 'is this k useful?'.

DO: report convergence and model-selection evidence separately. DO NOT keep training merely because k is poor, or mistake n_init=10 for ten iterations.

# Validation chooses; test checks once

Ordinary KMeans does not use validation scores inside its assignment/update loop

| Stage | Rows used | Example in this lesson |
|---|---|---|
| Declare the search | Before examining test results | Candidates k=2,3,4,5,6; highest validation silhouette; smaller k on an exact tie. |
| Fit candidate models | 360 training customers | Fit ONE training scaler; fit each k with 10 starts in that geometry. |
| Compare candidates | 120 validation customers | Transform using that scaler; predict with each frozen candidate; calculate silhouette. |
| Select and freeze settings | Training + validation evidence | The declared rule selects k=3. Elbow/stability/profiles support interpretation; they do not secretly override the rule. |
| Refit selected model | 480 development customers | Fit a NEW scaler and selected-k model on training + validation. |
| Assess final model | 120 untouched test customers | Transform, assign, and report distance/separation. Do not tune against these results. |

Validation customers DO NOT move candidate centers. Validation helps you choose settings BETWEEN fits. Changing k means fitting another candidate; it is not another iteration of the same model.

Other settings and feature/scaling choices can also be compared on development evidence. Predeclare a bounded search; repeated informal tuning can overfit validation too.

## What if evidence disagrees?

If the elbow suggests 3 but silhouette favors 2, report the disagreement. For this reproducible exercise, follow the declared rule. For a real project, investigate geometry, profiles, and action costs; revise a justified development-only plan and seek fresh confirmation.

DO NOT try k values until the TEST score looks good, hide unsuccessful searches, or claim any selected k proves the number of biological species. Without known labels, these scores assess geometry, not species identification.

# Our larger case: 600 synthetic records

A real executed small-data lab, not a claim about actual customers

Features are orders per month and average basket in dollars. The generator creates 600 records with deliberately clear structure and returns no target. We do not fit or select using its hidden construction groups.

| Pool | Rows | Purpose |
|---|---|---|
| Training | 360 | Fit scaler and candidate centers |
| Validation | 120 | Choose k and examine stability |
| Test | 120 | Final frozen-model assessment |
| Development | 480 | Train + validation refit after selection |

Split indices with seed 42: first 20% test, then 25% of development for validation. No target is available for stratification. Verify no overlaps and full coverage before scaling.

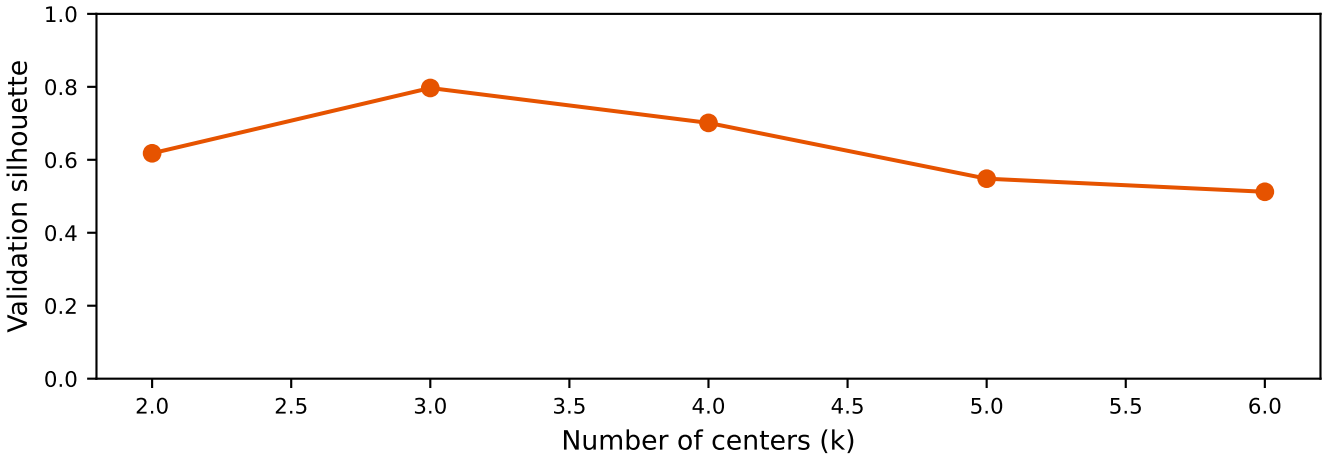
Fit the scaler only on training. Fit each candidate in that fixed scaled geometry. Then freeze choices, refit a NEW scaler and centers on development, and evaluate test once.

This split answers future-assignment questions. Descriptive clustering of a fixed dataset can use all rows, but then cannot claim independent generalization evidence. Repeated customers need group holdouts; future deployment may need time holdouts.

# Read the k-selection results

Numbers below are generated by the supplied lab

| k | Train mean d squared | Validation mean d squared | Val silhouette |
|---|---|---|---|
| 2 | 0.9944 | 0.8797 | 0.6178 |
| 3 | 0.1473 | 0.1478 | 0.7969 |
| 4 | 0.1156 | 0.1162 | 0.7011 |
| 5 | 0.0945 | 0.0956 | 0.5482 |
| 6 | 0.0835 | 0.0887 | 0.5123 |




The declared rule selects k=3 in this run. Lowest training error is NOT the rule. A k with extra centers may split an already coherent pattern without improving separation.

Scores are in standardized geometry. Neither 0.8 silhouette nor 0.1 mean squared distance means 80% or 90% correct. There are no known segment labels here.

# What do the metrics actually mean?

Always identify the numerator, denominator, and coordinate system

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

a = mean distance from one record to other members of its own cluster. b = the smallest mean distance to the records of another cluster, not just the distance to its center.

$$a = 0.4, \quad b = 1.0: \qquad s = \frac{1.0 - 0.4}{1.0} = 0.6$$

Average record silhouettes for the overall score. Near 1: separated; near 0: overlap; below 0: a different group may be closer on average.

Silhouette is defined when the assessed labels contain at least 2 groups and fewer groups than records. An isolated singleton has special handling in common implementations. Not every k/assignment yields a usable score.

| Metric | Meaning and limitation |
|---|---|
| WCSS (J / inertia) | Total squared distance to assigned centers. Depends on row count and scaling; smaller alone does not choose k. |
| Mean squared distance J/n | Average squared distance to assigned center. Same geometry needed for comparisons. |
| Silhouette | Within/between-group separation, not correctness or business value. |
| Stability ARI | Agreement between partitions, invariant to swapped cluster IDs; not accuracy. |

sklearn KMeans.score is NEGATIVE total squared distance, not classification accuracy. This lab negates it and divides by the number of assessed rows.

# Validate stability and interpret profiles

Repeat fits can agree without proving the groups are real

Fit five single-initialization models using seeds 0-4 on training rows. Assign the SAME validation records with each model. Their ten pairwise ARI scores average 1.0000 in this run.

ARI=1 means identical partitions up to renaming. Around 0 means chance-level agreement under its adjustment; negative values are possible. High agreement on this easy simulation is not evidence that real customers have these exact types.

| Cluster ID | Development count | Orders/month | Basket ($) |
|---|---|---|---|
| 0 | 157 | 14.99 | 35.20 |
| 1 | 152 | 2.02 | 24.83 |
| 2 | 171 | 7.95 | 74.92 |

These are inverse-transformed final-development centers. Describe relative behavior such as 'frequent, smaller baskets', not moral or demographic identities. IDs may change after a refit.

A center is a representative average, not necessarily a customer. Inspect within-group spread and size, not only the average. Test a proposed action ethically rather than assuming clustering caused a business improvement.

# When are we really done?

Convergence, hyperparameter selection, and deployment are different

Within full KMeans: assignments may stop changing, or center shift falls below its tolerance threshold. max_iter limits work. scikit-learn scales its center-shift tolerance using data variability; it is not an accuracy target or a requested inertia.

n_iter_ reports the retained run. Hitting a cap merits checking movement/objective, more budget, and repeated starts on development data. Early stopping can leave centers not exactly equal to means of the final reassigned labels.

Across starts: n_init chooses the retained full-KMeans run by training inertia. Across k: our validation rule chooses k. No test score participates in either decision.

| Final test model | Mean d squared | Silhouette |
|---|---|---|
| full_kmeans | 0.129188 | 0.798762 |
| streamed_minibatch | 0.129188 | 0.798762 |

The test scores assess geometry for similar new records, not true-segment correctness. The mini-batch result may equal the full result on this easy case; neither equality nor a speed advantage is guaranteed.

After evaluation, deployment may refit on more unlabeled feature rows. The old held-out score then describes the old model; obtain fresh evidence for the new model.

# What changes at 85 million rows?

Hypothetical arithmetic only; there is no huge dataset supplied

Suppose N=85 million customers, d=2 features, k=3. Full Lloyd compares each row with three centers per assignment pass. It does NOT compare every customer with all 85 million other customers.

| Quantity | Calculation |
|---|---|
| Raw float64 features | 85,000,000 x 2 x 8 bytes = 1.36 GB decimal |
| Point-center comparisons / pass | 85,000,000 x 3 = 255,000,000 |
| Across 20 passes | 255,000,000 x 20 = 5.1 billion comparisons |
| Ten complete full-KMeans restarts | 5.1 billion x 10 = 51 billion comparisons |
| Coordinate differences / pass | 255 million x 2 = 510 million; not the same unit |

The 1.36 GB excludes IDs, copies, labels, buffers, scaler data, and initialization. Actual passes and convergence vary. More features, k candidates, restarts, and a poor storage layout add cost.

Standard full KMeans.fit is not an automatic out-of-core iterator. Elkan can allocate N-by-k bounds. Small center storage (k by d) does not mean the fitting process uses only that much memory.

No universal row ceiling: benchmark memory, IO, runtime, initialization, and assessment cost on representative data. Distributed or GPU systems can help, but do not change the objective or prove usefulness.

# Blocks versus genuine mini-batch learning

Unlike basic KNN, mini-batch K-means really updates learned parameters

### Full Lloyd in blocks

Keep centers fixed for the whole pass. Read blocks, assign rows, and accumulate a sum vector and count per center. After ALL blocks, divide global sums by counts and update centers. This reproduces the full assignment/update step, apart from rounding and explicit empty-cluster handling.

### MiniBatchKMeans

Updates centers after each mini-batch using running counts/averages, not a full-pass reassignment. It is an approximate optimization route with different convergence behavior. It is not running independent KMeans on each block.

At 100,000 rows per chunk, 85 million rows takes 850 chunks per pass. Mini-batch updates need not decrease full-data J after every step. Evaluate progress on a fixed development sample, not the test set.

MiniBatchKMeans has partial_fit; full KMeans does not. Repeated fit calls replace a model. batch_size in fit controls its batching; with partial_fit, YOU supply the batch and control passes/stopping. The first batch must support the requested k.

MiniBatchKMeans n_init selects initialization candidates, not ten full training restarts like KMeans. Its fit has max_iter and early-stop controls tol/max_no_improvement; caller-driven partial_fit does not enforce a global stopping loop.

Other mini-batch settings include batch_size (work per update) and reassignment_ratio (handling low-count centers). Their effects need development-only checks; defaults are not universal best settings. Standard KMeans uses squared Euclidean geometry, not an arbitrary distance-metric hyperparameter.

# A realistic streaming plan

Keep one feature geometry and bounded monitoring

1. Check units, missingness, duplicate customers, coverage, outliers, and privacy. Define a deployment-matching split or a clearly descriptive scope.

2. First training-only pass: use StandardScaler.partial_fit over chunks to accumulate GLOBAL means and variances. Do not give each block a different scaler.

3. Freeze that scaler. Second pass: transform chunks with the fixed statistics. Initialize centers from a representative development/training sample, not only the first site's rows.

4. Update MiniBatchKMeans with controlled passes and batches. Check distance trends and center movement on a fixed development-only monitor sample; predeclare patience and maximum work. A budget exit is not convergence.

5. Compute cluster counts and distance summaries by streaming predictions with fixed centers. For silhouette, use a representative bounded sample, for example 2,000 records, not a giant all-pairs matrix.

Silhouette involves pairwise distances within the assessed sample: roughly quadratic work. A full 85-million by 85-million matrix is infeasible. A 2,000-row sample contains about 4 million pairwise entries; report sampling uncertainty and per-cluster coverage.

6. Freeze choices, then evaluate held-out data once if future assignment is the goal. Monitor drift, empty/tiny groups, distance distributions, latency, subgroup consequences, and action usefulness. New data does not silently refit the model.

# Connect the lesson to Python

A fit learns centers; predict only assigns

The runnable script and notebook use these exact concepts. X_train contains two numeric features; there is no y argument with observed customer labels.

## Preprocessing and fitting

```
scaler = StandardScaler().fit(X_train)
Z_train = scaler.transform(X_train)
model = KMeans(n_clusters=k, init='k-means++',
n_init=10, max_iter=300, tol=1e-4,
random_state=42, algorithm='lloyd')
model.fit(Z_train)
```

## Assignment and interpretation

```
Z_new = scaler.transform(X_new)
groups = model.predict(Z_new)
raw_centers = scaler.inverse_transform(
model.cluster_centers_)
```

Save preprocessing and centers together. Never pass raw dollars to a model fitted on z-scores or report a z-score center as dollars. predict does not update either component.

Read kmeans_lab.py and run the notebook cells top to bottom. The script exports four CSVs and four figures. Make each prediction before the notebook shows its self-check.

Before final test evaluation, freeze the selected k and all geometry choices, then fit a NEW scaler and KMeans on the 480 development rows. The displayed code above illustrates one fitting stage, not permission to refit on test rows.

# Limits, next steps, and your lab

Useful groups require evidence beyond an attractive scatterplot

| Failure mode | What to investigate |
|---|---|
| Curved or irregular groups | Density-based methods such as DBSCAN/HDBSCAN; noise and parameter choices still matter. |
| Outliers and skew | Audit records, justify transformations/robust preprocessing; do not delete inconvenient people. |
| Unequal densities or broad overlap | Other objectives or mixture models; no universal winner. |
| Categorical codes or many irrelevant features | Choose meaningful representations; numeric IDs are not distances. Consider validated feature reduction. |
| No meaningful separation | Report uncertainty or decline to operationalize the segments. |

Open the guided notebook, make predictions before running cells, and complete the independent homework. You should be able to explain each update, stopping condition, setting, metric, and scale decision without reading an output number aloud.

Official references: scikit-learn KMeans, MiniBatchKMeans, silhouette_score, adjusted_rand_score, and clustering user guide. Exact URLs are in the folder README. The lab records measured small-data results; all large-scale costs are hypothetical.